



Java组件化开发实战

北京理工大学计算机学院
金旭亮

组件化开发概述



现在的软件系统功能越来越复杂，规模也越来越大，为了应对这种挑战，人们将“老系统”中可以重用的代码抽取出来单独构建为“**组件 (Component)**”，开发新系统时，直接使用这些现成的组件而不是一切都重新再来，是“**组件化开发 (CBD: Component Based Development)**”的基本思想。

Java应用程序开发时，最普遍使用的组件就是jar包，JDK的各个组件，基本上都是以jar包的形式提供给Java开发者的。

下面我们通过一个例子一步步地展示以重用为目的的Java代码演化过程：

原始程序

```
public class ReverseString1 {  
  
    // 示例程序功能：将一个字符串倒序输出  
    public static void main(String[] args) {  
        String str = "abcd";  
        StringBuffer buff = new StringBuffer();  
        for (int i = str.length() - 1; i >= 0; i--)  
            buff.append(str.charAt(i));  
        System.out.println(buff.toString());  
    }  
}
```

示例项目：ExploreJavaPackage

最初想法：

示例代码中，将所有代码都放到了main()方法中，并且混杂了数据处理代码（实现字符串倒序）和输出代码（println），能不能将两者分开以便让代码更易于理解？

重构：方法抽取

```
public class ReverseString2 {  
  
    public static void main(String[] args) {  
        String str = "abcd";  
        System.out.println(reverseString(str));  
    }  
  
    // 方法功能：将一个字符串倒序输出  
    private static String reverseString(String inputStr) {  
        StringBuffer buff = new StringBuffer();  
  
        for (int i = inputStr.length() - 1; i >= 0; i--)  
            buff.append(inputStr.charAt(i));  
  
        return buff.toString();  
    }  
}
```

原问题解决，又发现新问题：

在同一个项目中，倒序输出的功能在好几个地方都用到，总不能把reverseString()方法的源代码在每个类中都Copy & Paste一遍吧？

重构：类的引入

```
package cn.edu.bit.cs.utils;
/**
 * 此类封装一些字符串操作的功能
 */
public class MyStringUtil {
    // 方法功能：将一个字符串倒序输出
    public static String reverseString(String inputStr) {
        StringBuffer buff = new StringBuffer();

        for (int i = inputStr.length() - 1; i >= 0; i--)
            buff.append(inputStr.charAt(i));

        return buff.toString();
    }
}
```

创建一个新的包，用于集中保存将要重用的类

技术要点：

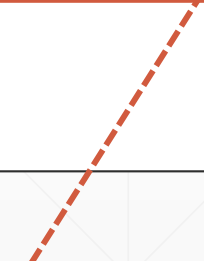
将需要重用的代码与其他代码“隔离”开来，放到独立的类中，现在，同一项目中就能很容易地重用这一功能了。

基于类的代码重用示例

```
//导入另一个模块包中的类
import cn.edu.bit.cs.utils.MyStringUtil;

public class ReverseString3 {

    public static void main(String[] args) {
        String str="abcd";
        System.out.println(MyStringUtil.reverseString(str));
    }
}
```



现在在同一个项目中，任何一个需要实现字符串倒序功能的地方，都可以直接调用MyStringUtil.reverseString()方法，重用代码很方便。

新的问题



如果我有一个新项目，里头也想调用字符串倒序功能，那该怎么办呢？

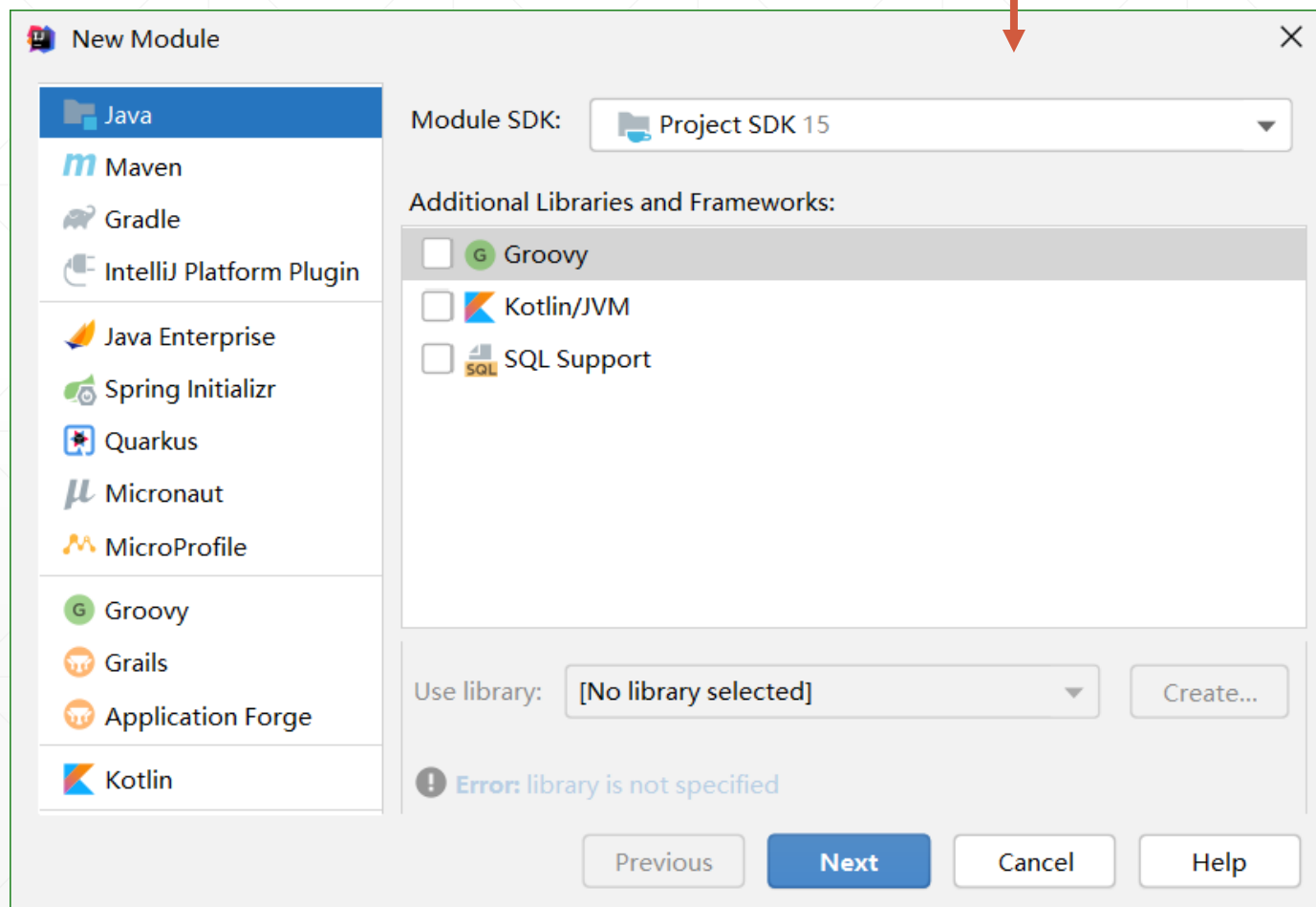
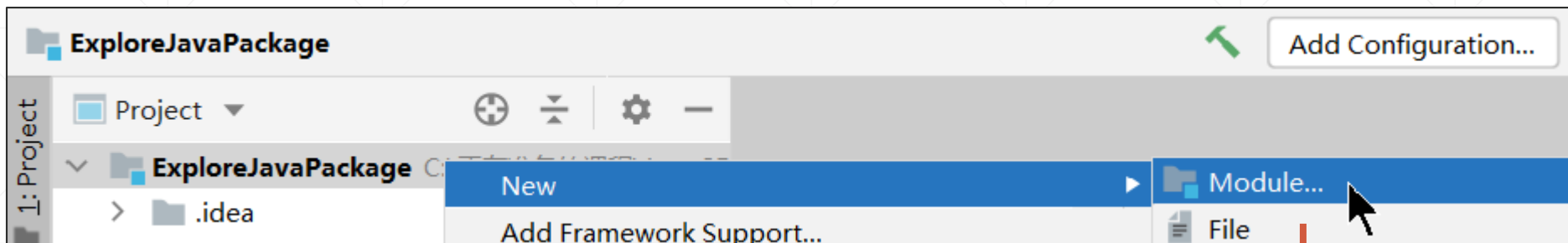
解决思路



将需要跨项目重用的代码，封装为一个单独的jar包，然后在新项目中将其添加进来，这样一来，只要有一个jar包，就可以直接使用jar包中的代码。

下面，让我们演示一下这个过程……

Project面板的项目结点上右击，从弹出菜单中选择“新建一个模块”命令，向项目中添加一个Java模块……



 New Module ✕

Module name:

MyJavaLibrary

Content root:

第2部分 Java面向对象编程基础\7 包与组件化开发\ExploreJavaPackage\MyJavaLibrary 

Module file location:

第2部分 Java面向对象编程基础\7 包与组件化开发\ExploreJavaPackage\MyJavaLibrary 

给模块起名: MyJavaLibrary

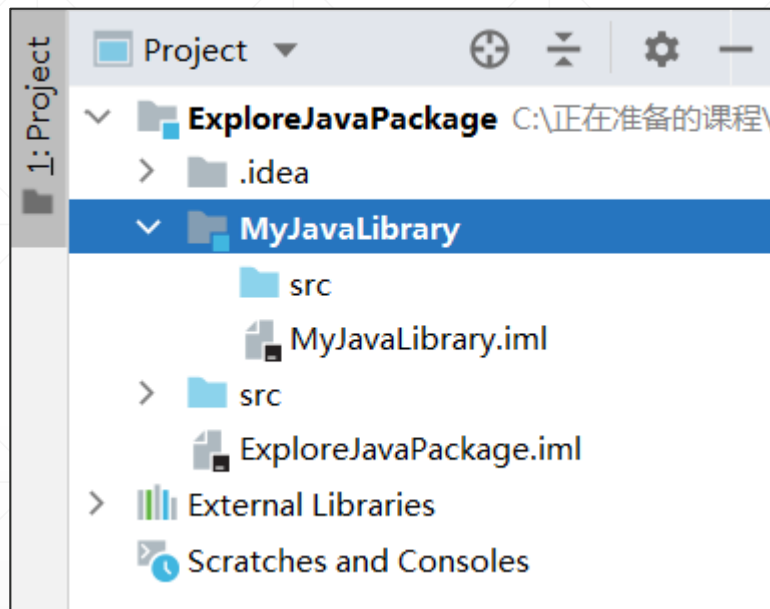
Previous

Finish

Cancel

Help

IntelliJ中新加的模块

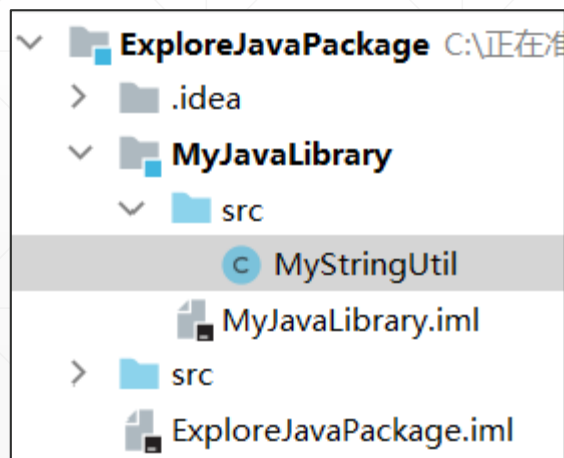
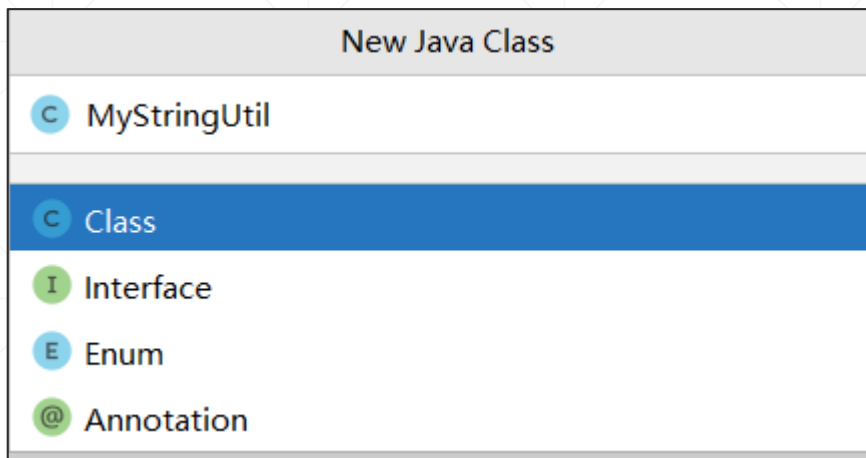


新加的模块出现在Project面板中，展开之后，可以看到它也有一个src文件夹，可以向其中添加相应的Java文件。



在IntelliJ中，每个“项目（Project）”都可以包容多个“模块（Module）”，模块是IntelliJ编译Java项目的基本单元。

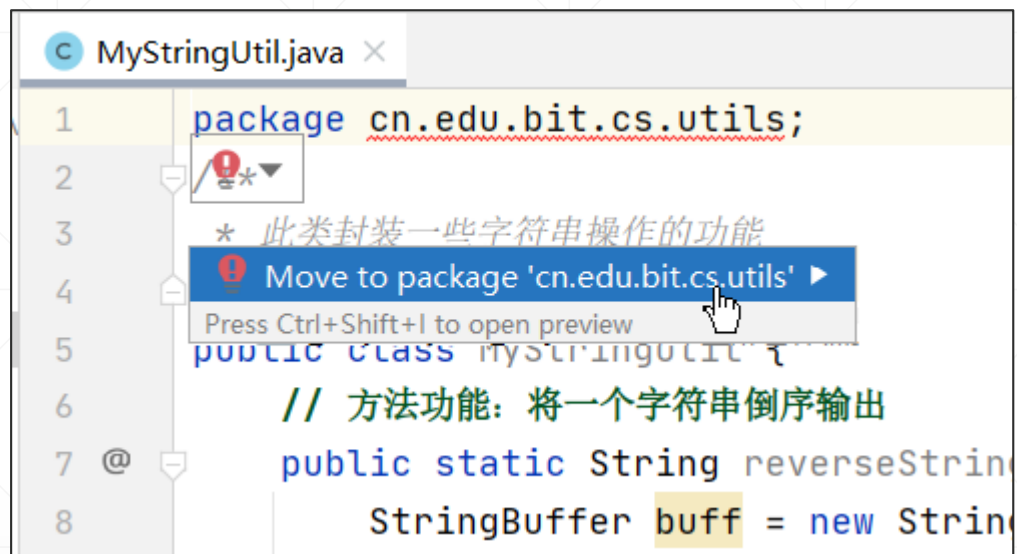
向MyJavaLibrary模块中添加Java类



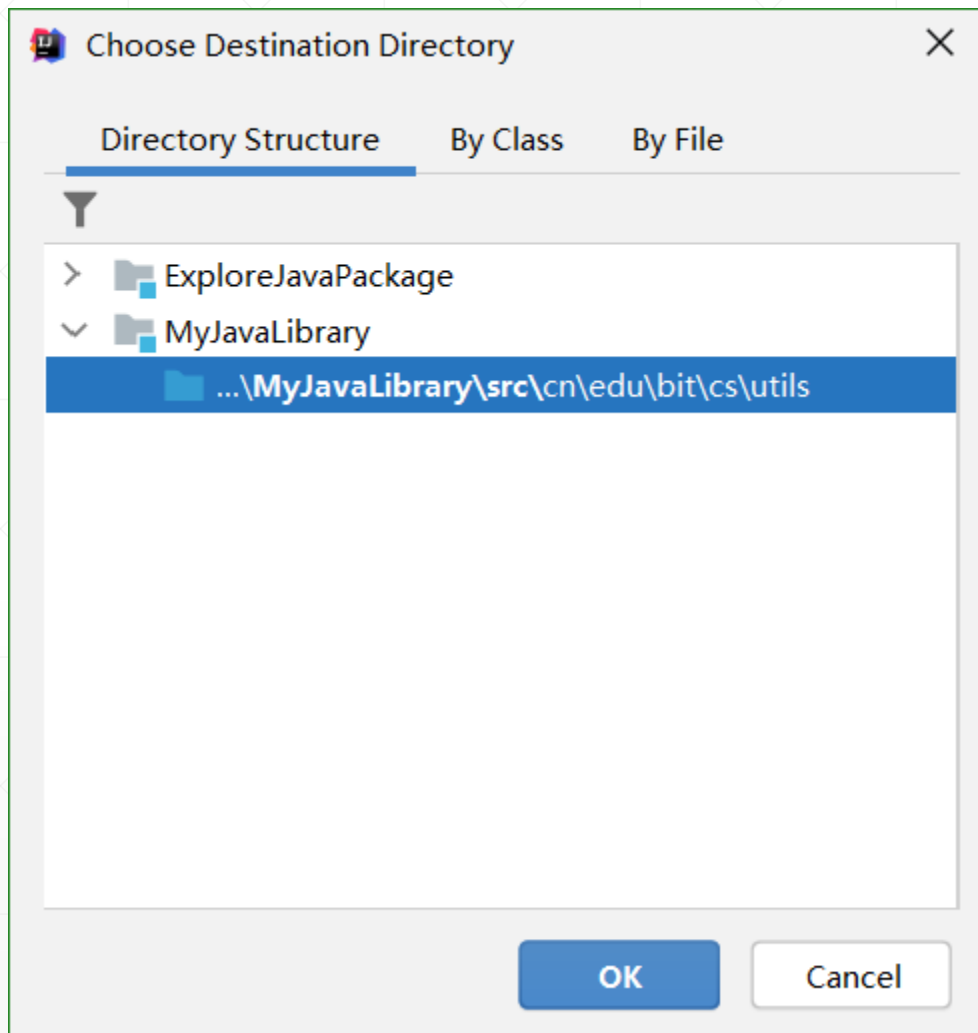
包的声明语句被IntelliJ打上了红色波浪线，因为它与文件所在的文件夹结构不一致

```
1 package cn.edu.bit.cs.utils;  
2 /**  
3  * 此类封装一些字符串操作的功能  
4  */  
5 public class MyStringUtil {  
6     // 方法功能: 将一个字符串倒序输出  
7     @ public static String reverseString(String inputStr) {  
8         StringBuffer buff = new StringBuffer();  
9  
10        for (int i = inputStr.length() - 1; i >= 0; i--)  
11            buff.append(inputStr.charAt(i));  
12  
13        return buff.toString();  
14    }  
15 }
```

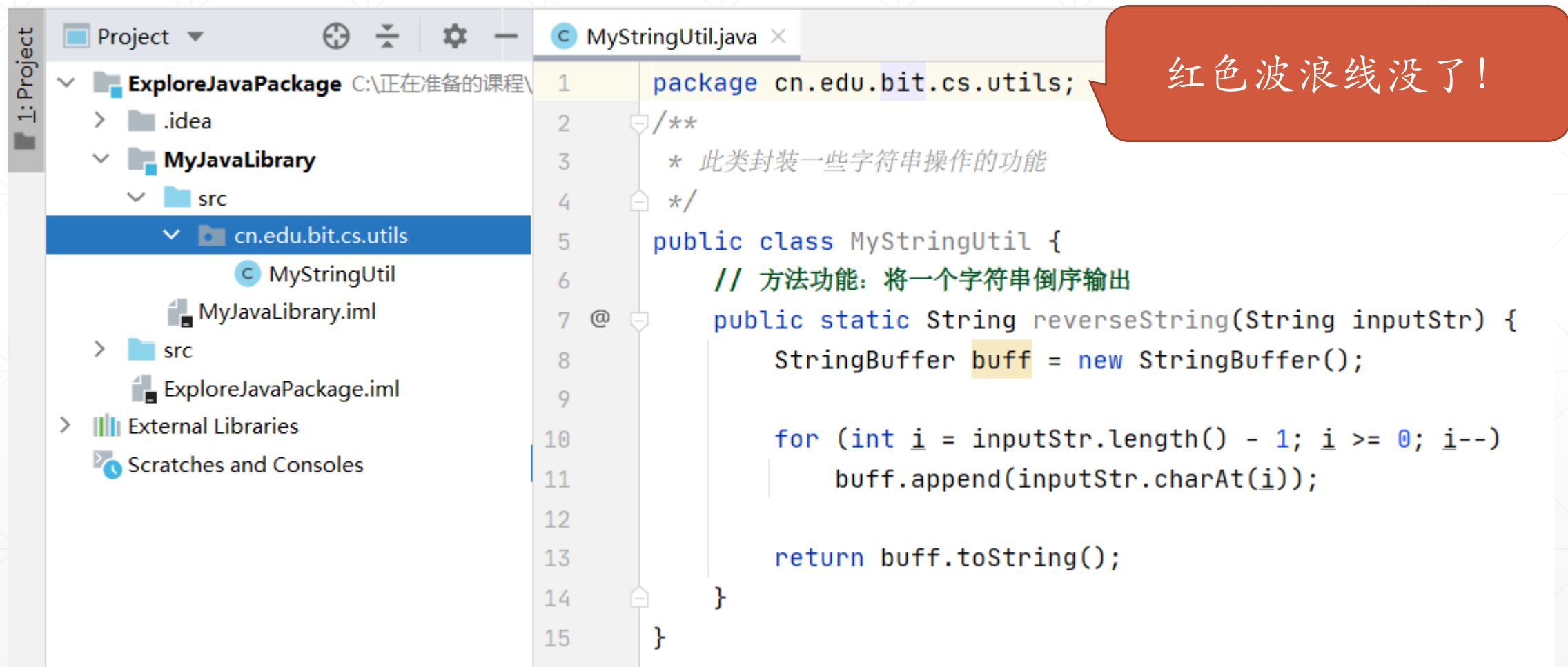
IntelliJ来帮助.....



IntelliJ提供了移动类到特定包中的“代码重构”功能，可以用它把MyStringUtil类移到指定的包中，IntelliJ会自动创建相应的文件夹。



现在可以顺利编译的MyStringUtil类.....

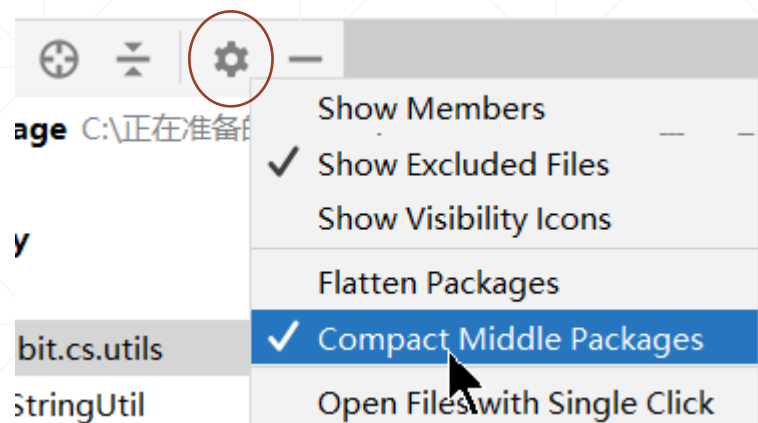


The screenshot shows an IDE window with the file `MyStringUtil.java` open. The left sidebar displays the project structure, including `ExploreJavaPackage`, `MyJavaLibrary`, and `cn.edu.bit.cs.utils`. The main editor area shows the following Java code:

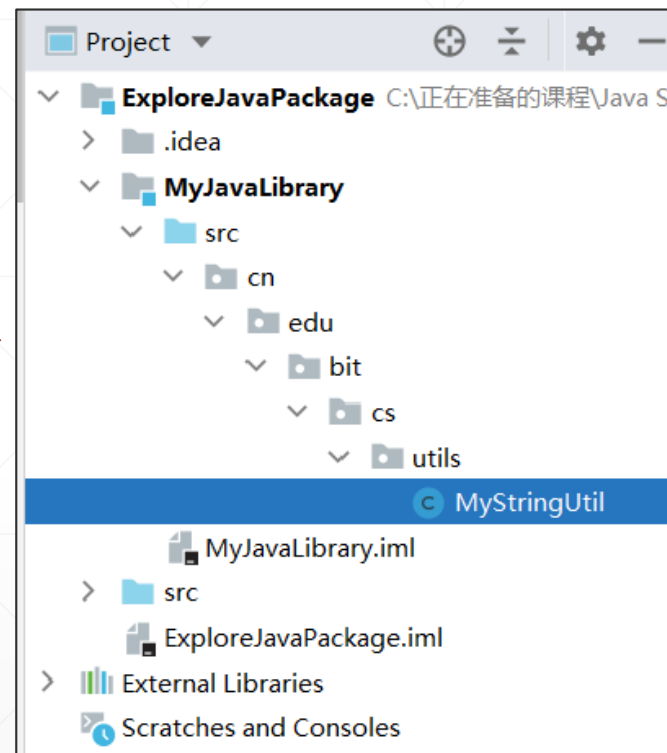
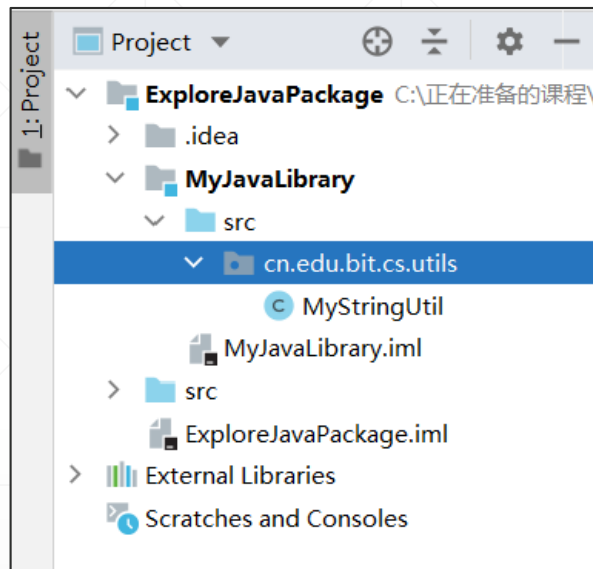
```
1 package cn.edu.bit.cs.utils;
2 /**
3  * 此类封装一些字符串操作的功能
4  */
5 public class MyStringUtil {
6     // 方法功能: 将一个字符串倒序输出
7     @ public static String reverseString(String inputStr) {
8         StringBuffer buff = new StringBuffer();
9
10        for (int i = inputStr.length() - 1; i >= 0; i--)
11            buff.append(inputStr.charAt(i));
12
13        return buff.toString();
14    }
15 }
```

A red callout bubble on the right side of the editor contains the text: 红色波浪线没了! (The red wavy lines are gone!).

多学一招!

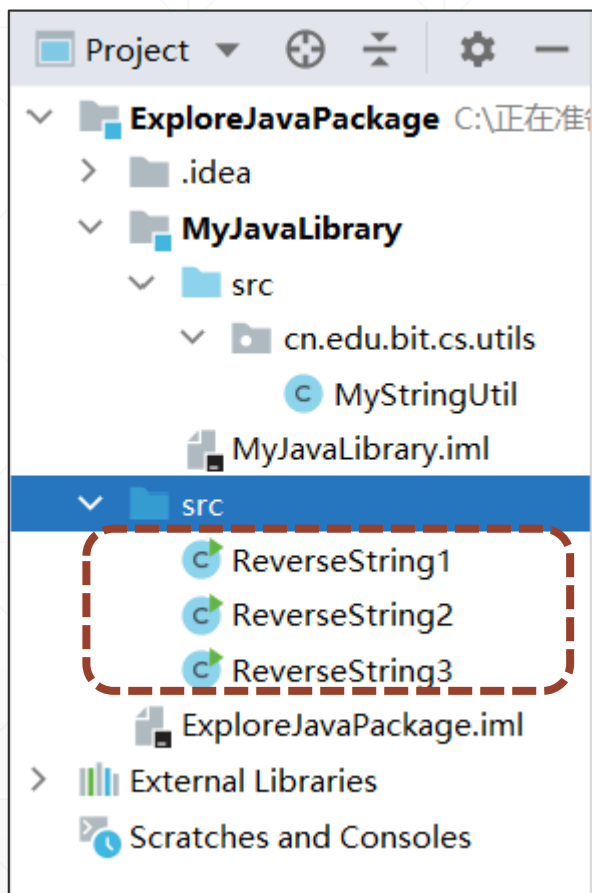


在Project面板中点击
“齿轮”图标，取消
“Compact...”选项

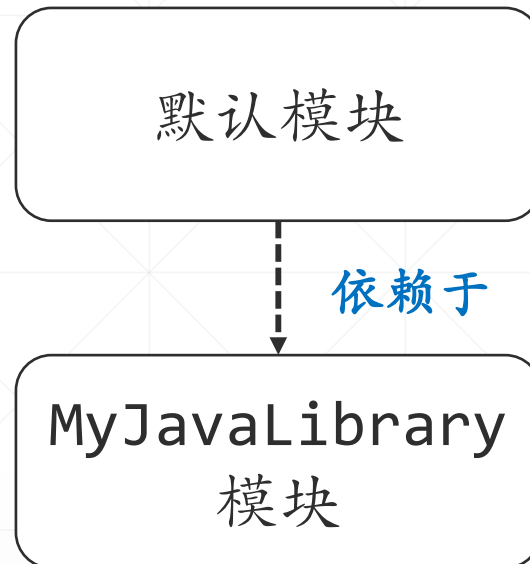


IntelliJ将按照包名显示完整的文件夹结构。

模块之间的依赖关系

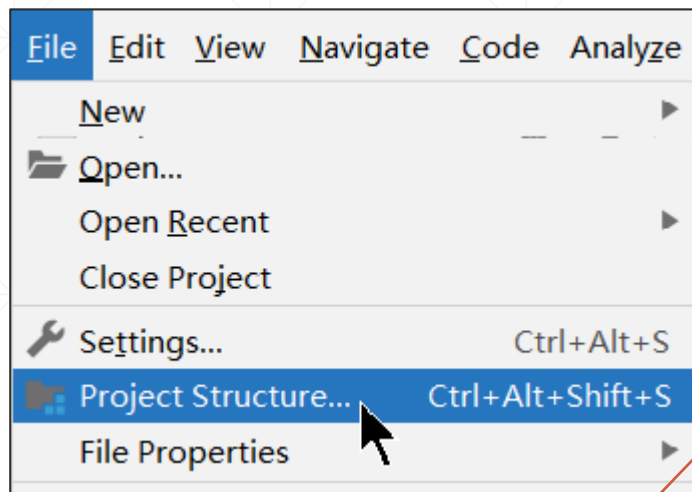


默认模块
中的代码

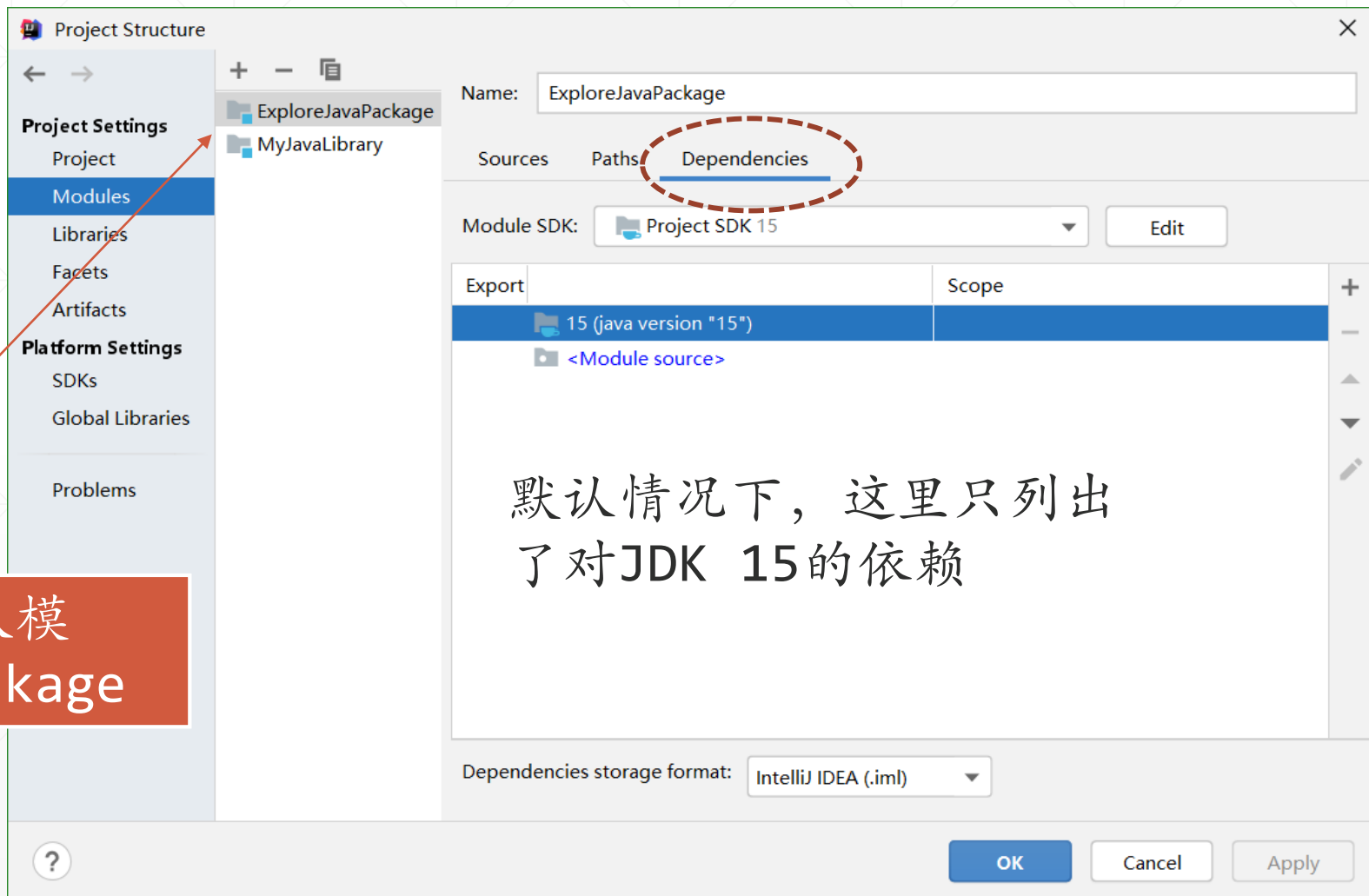


默认模块中的代码，如果要使用另一模块中的代码，需要在这两个模块之间建立依赖关系。

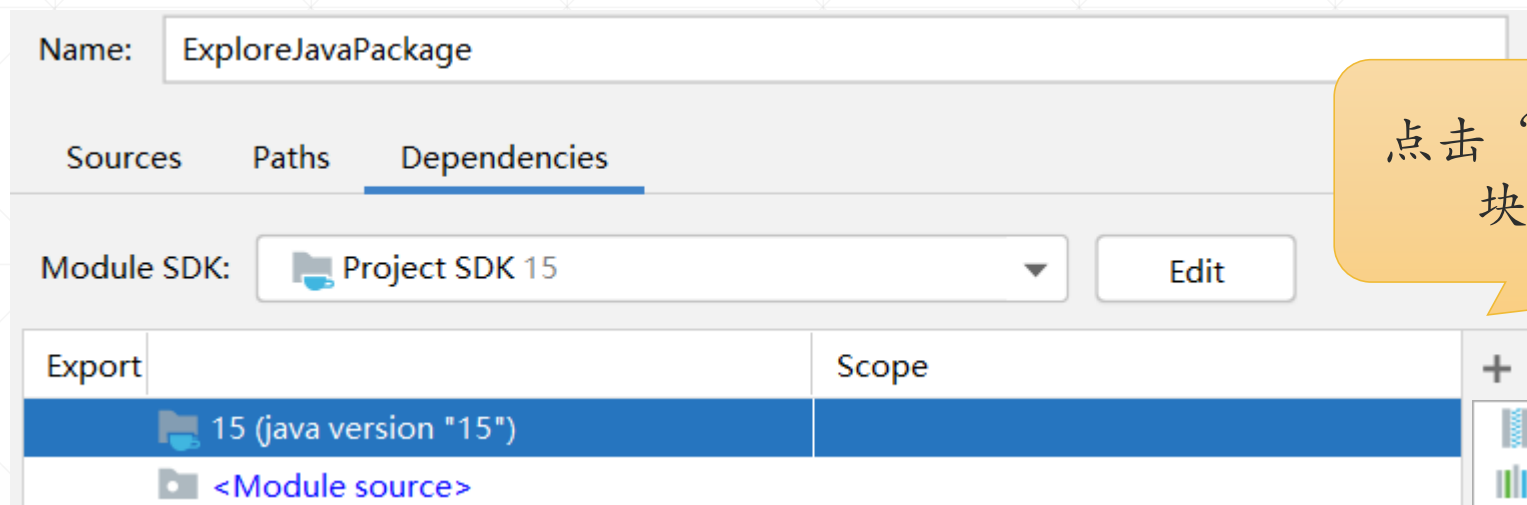
创建模块之间的依赖关系



在模块列表中选中“默认模块”——ExploreJavaPackage

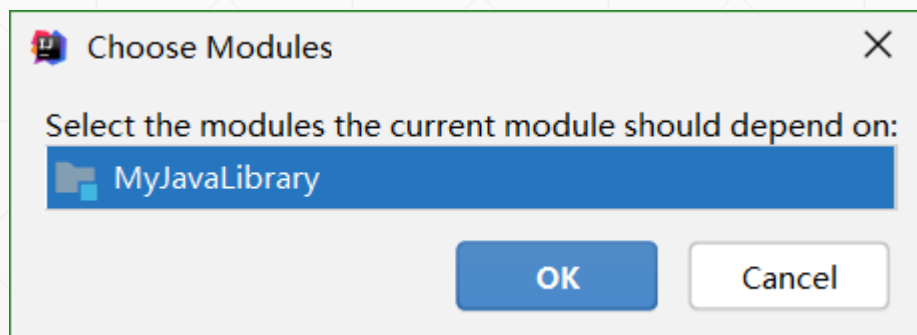


添加模块依赖

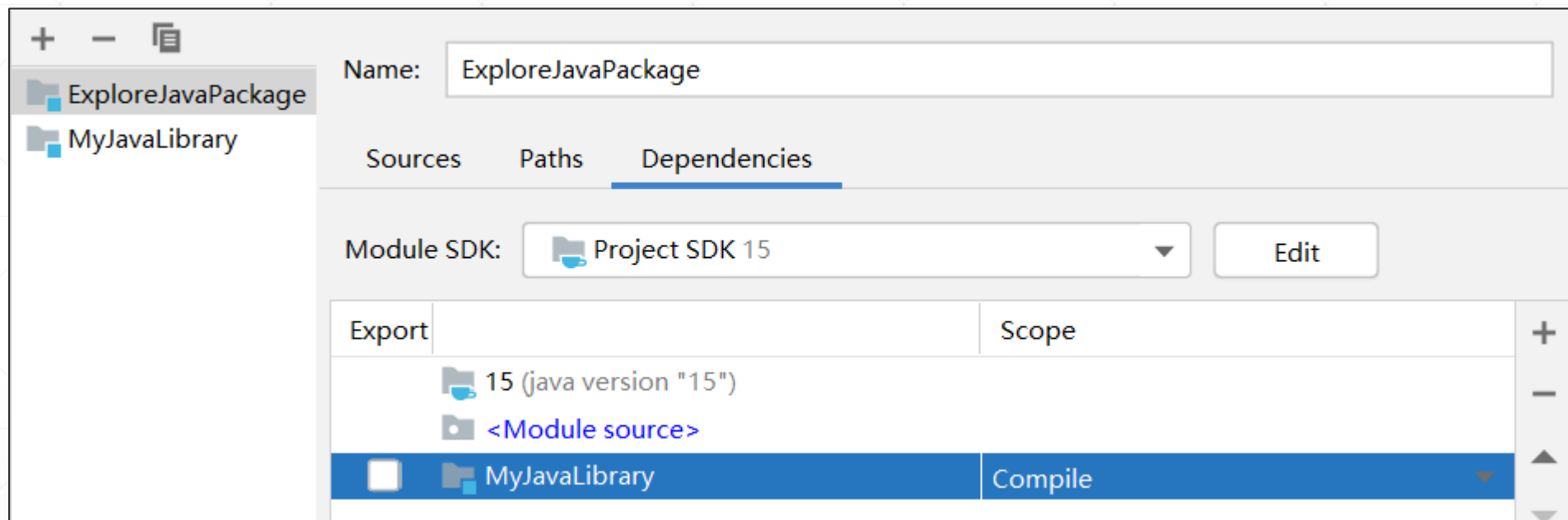


点击“+”号图标，可以给当前选中的模块，添加对其他模块的依赖关系

- 1 JARs or directories...
- 2 Library...
- 3 Module Dependency...

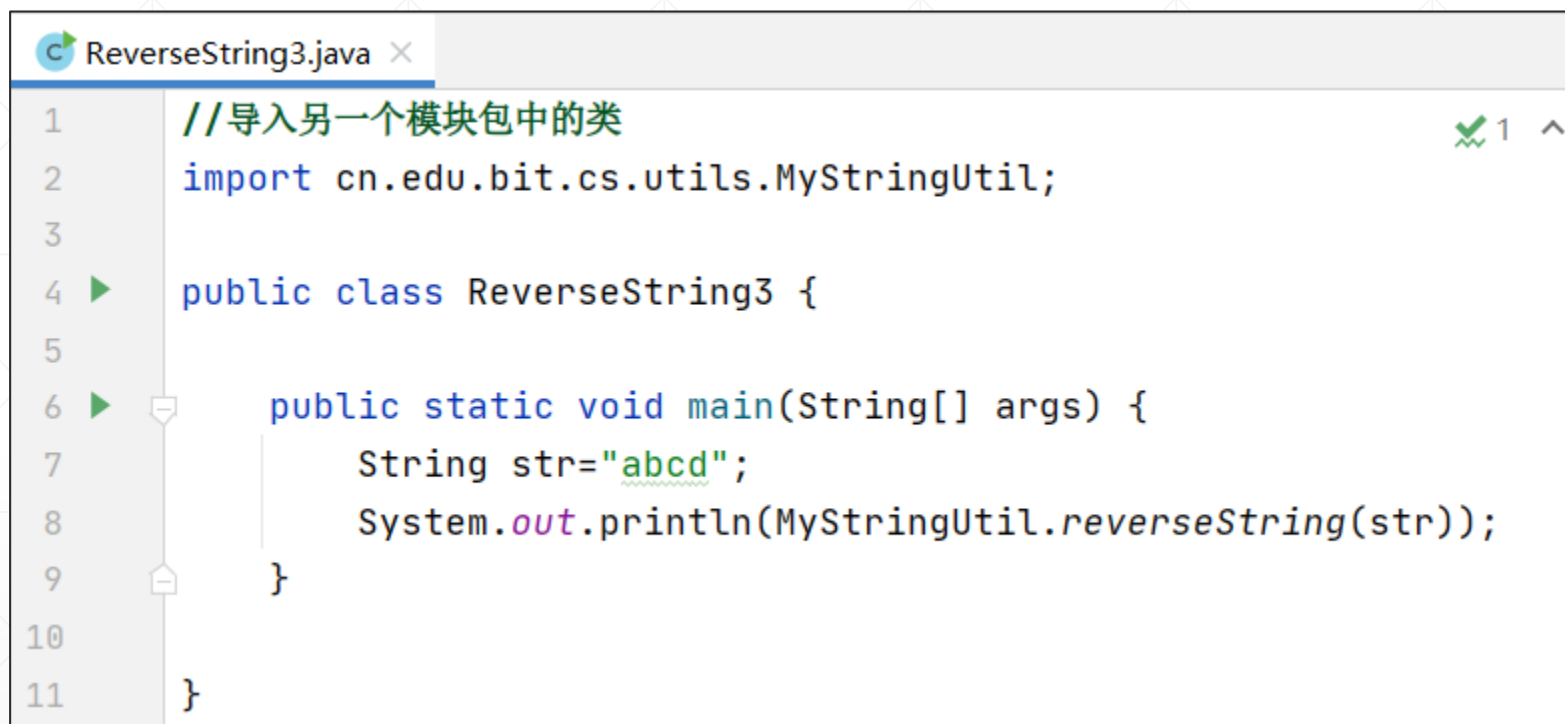


创建好模块依赖关系后的模样.....



ExploreJavaPackage模块创建好对MyJavaLibrary模块的依赖关系之后，就可以使用里面的MyStringUtil类了。

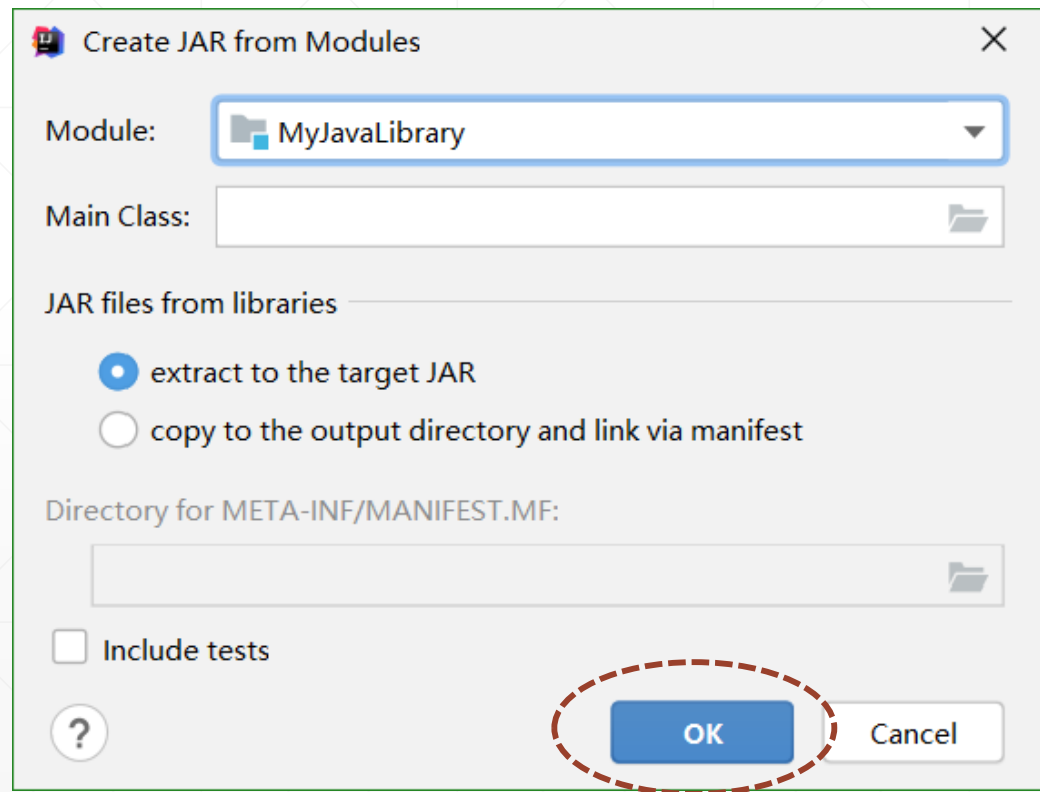
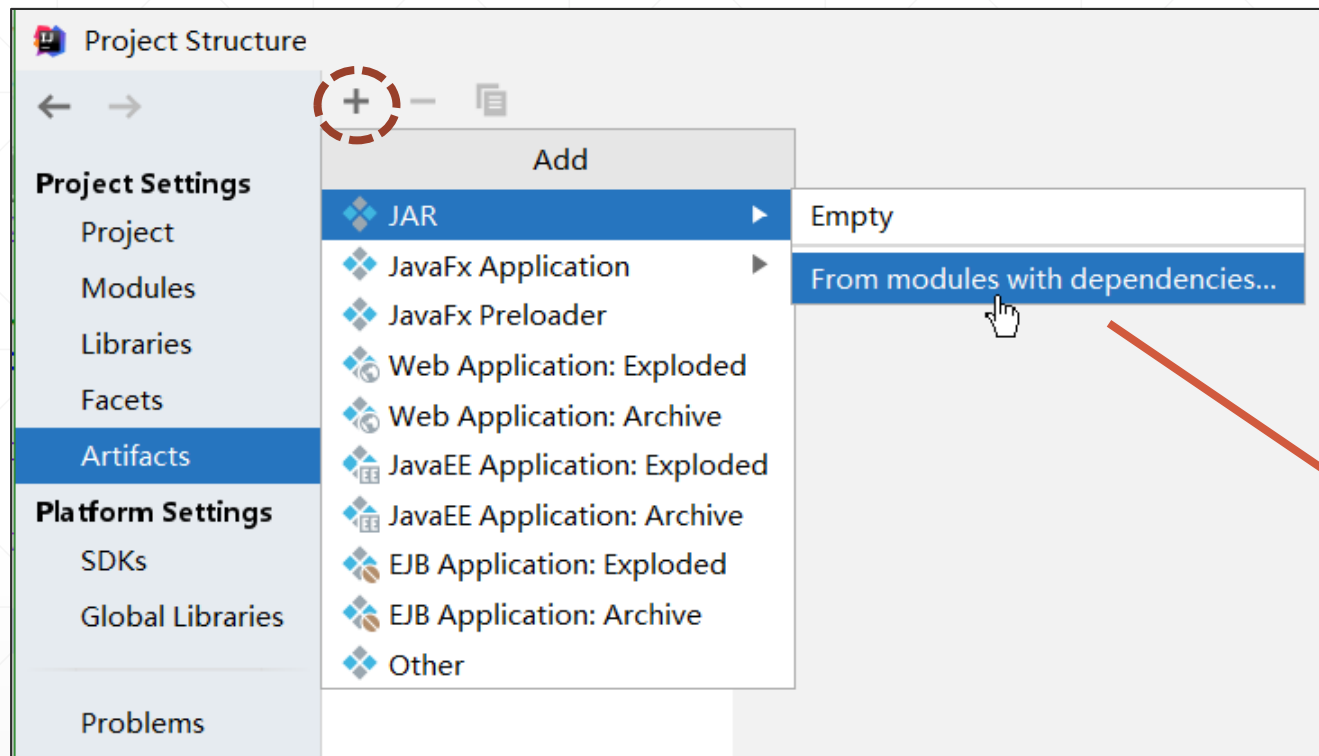
使用给模块添加依赖关系这种手段，可以方便地组织代码，实现同一项目内“**源代码级别**”的“**跨模块**”的代码重用。



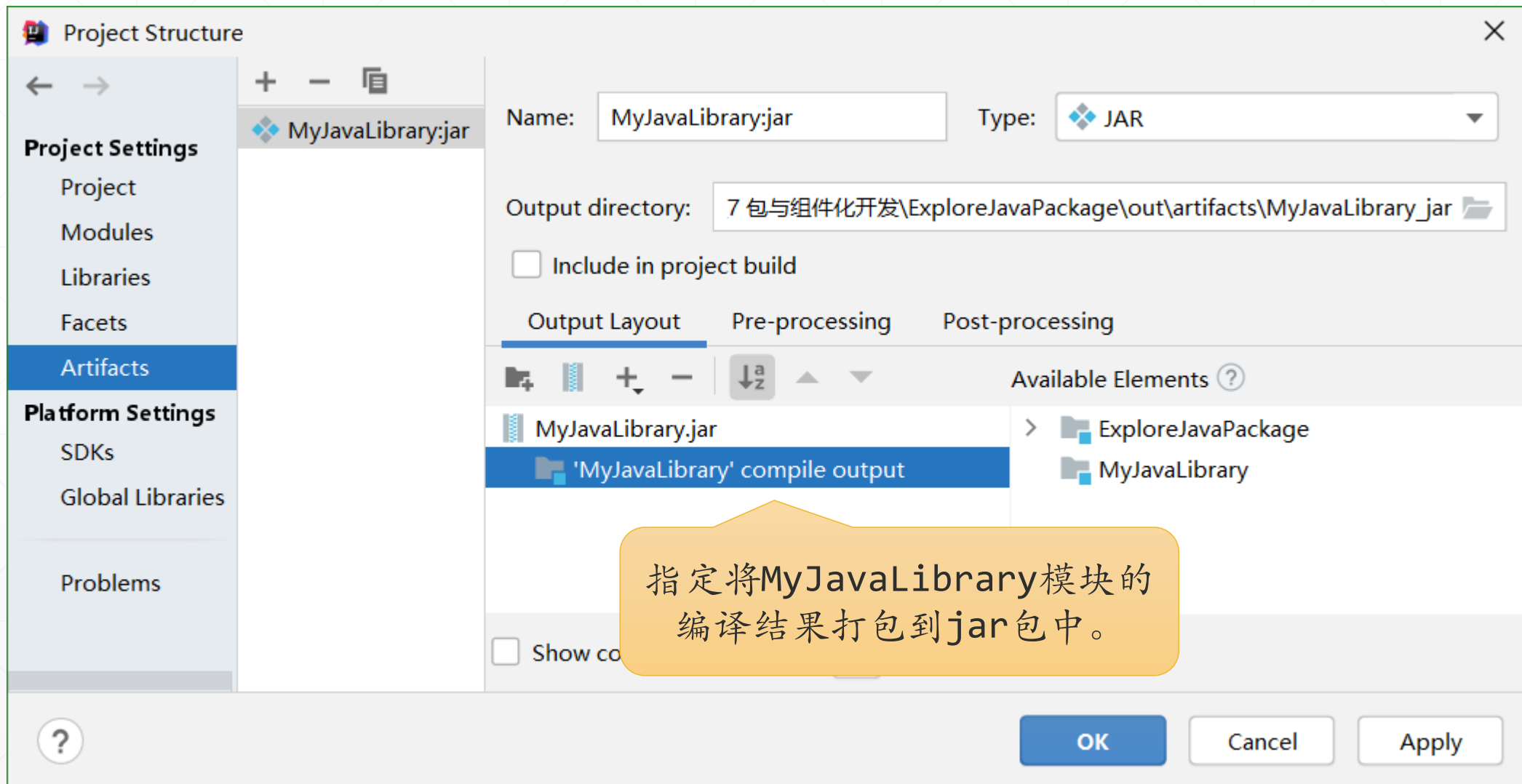
```
ReverseString3.java x
1 //导入另一个模块包中的类
2 import cn.edu.bit.cs.utils.MyStringUtil;
3
4 public class ReverseString3 {
5
6     public static void main(String[] args) {
7         String str="abcd";
8         System.out.println(MyStringUtil.reverseString(str));
9     }
10
11 }
```

更进一步，如果希望这些代码能“**跨项目**”重用，则可以将MyJavaClassLibrary模块导出为jar包，从而被另外的项目所重用。

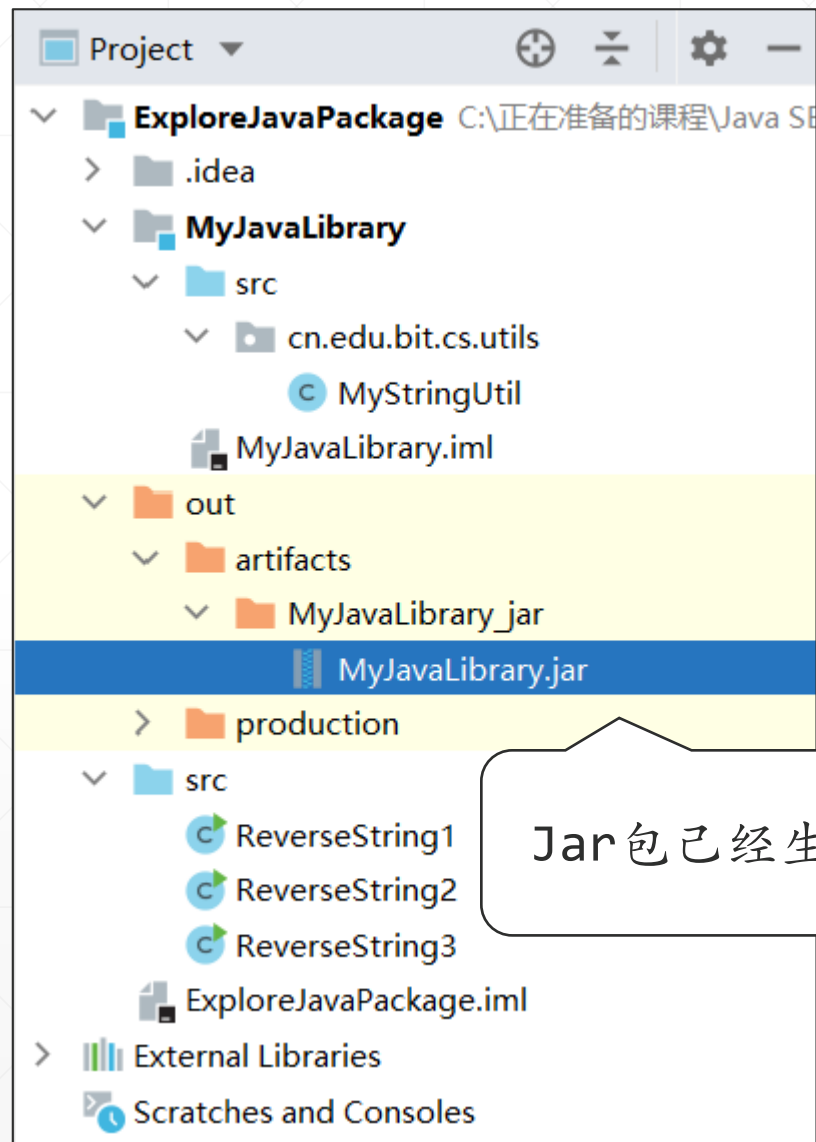
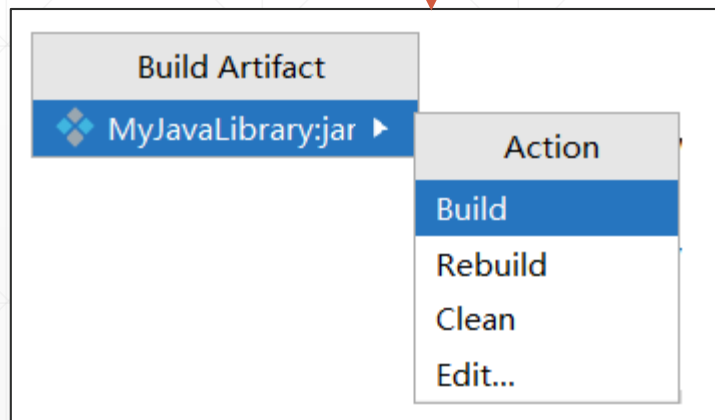
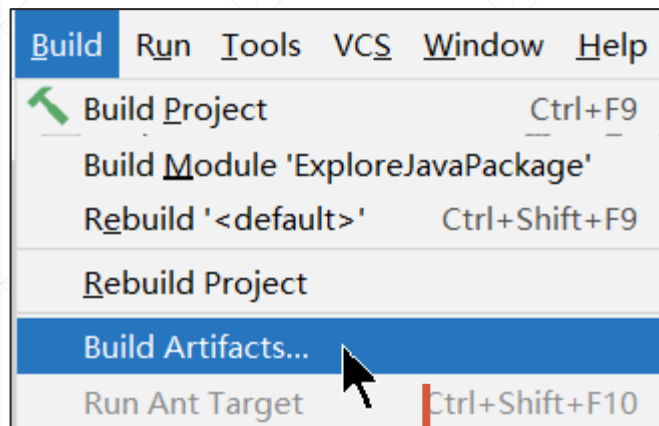
准备生成Jar包



配置完成之后的截图



生成jar包



Jar包已经生成!

New Project ✕

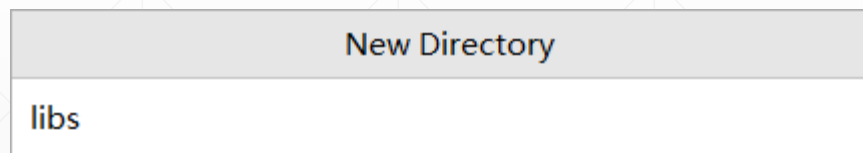
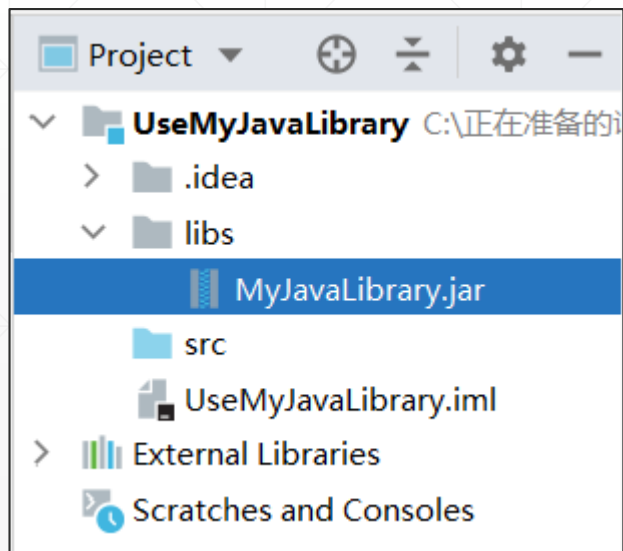
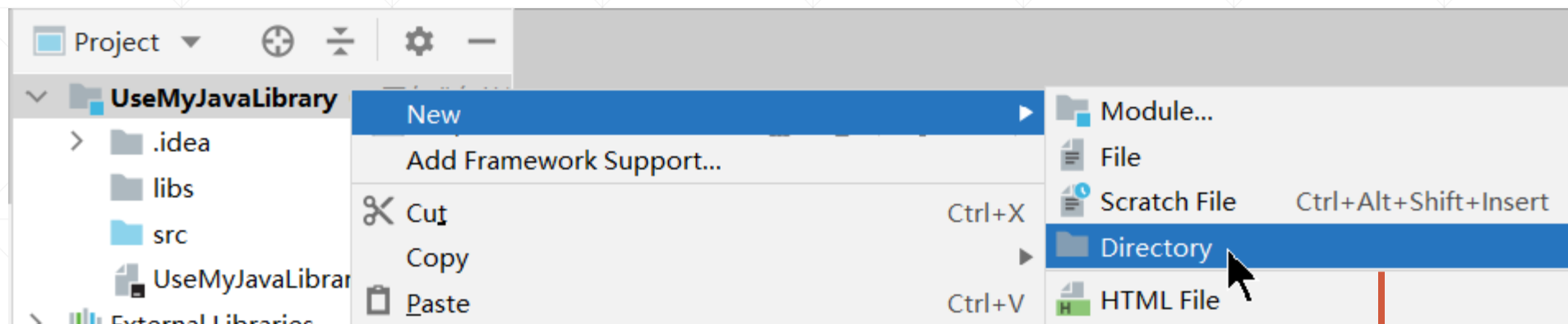
Project name:

Project location: ...

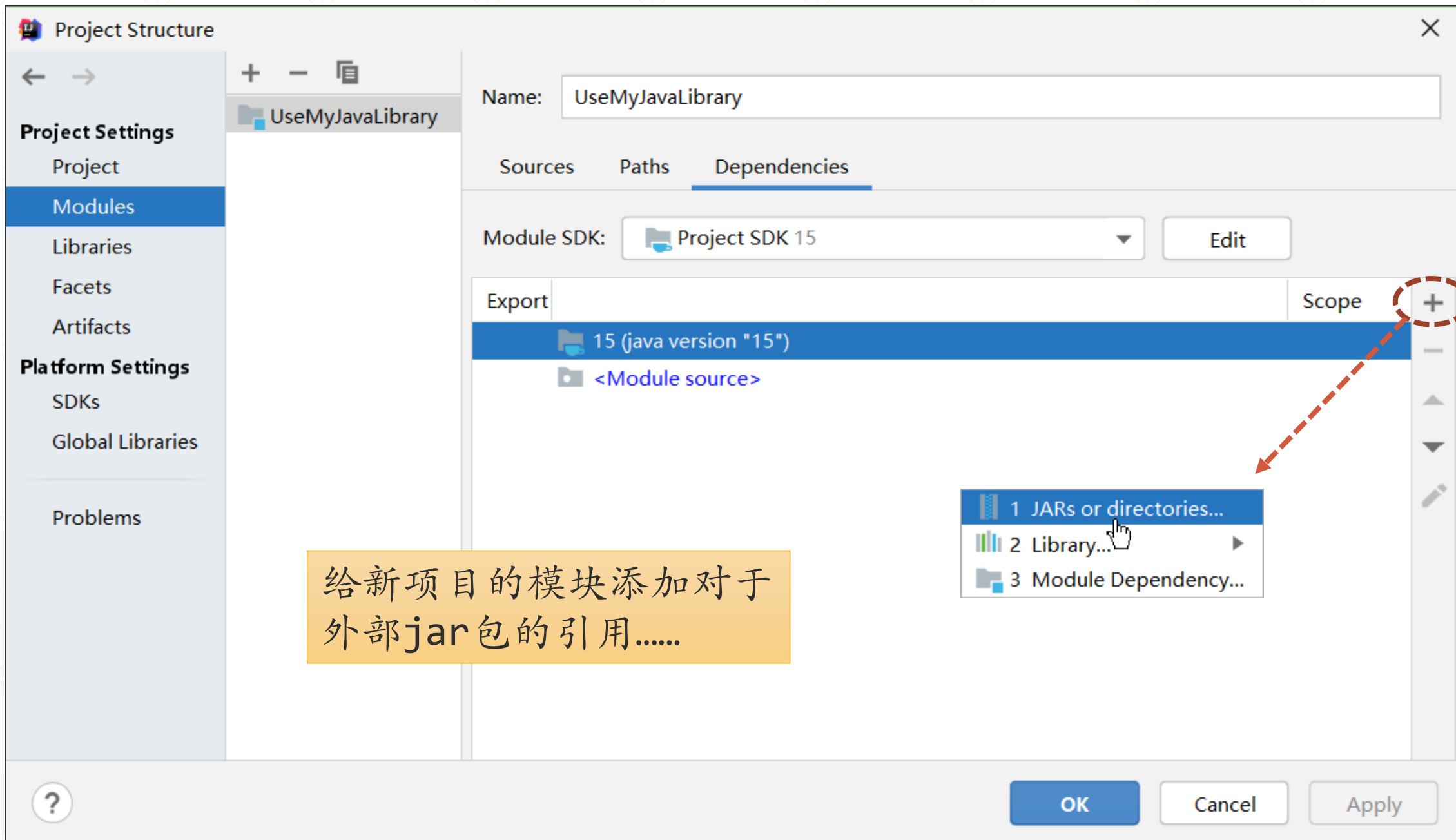
► More Settings

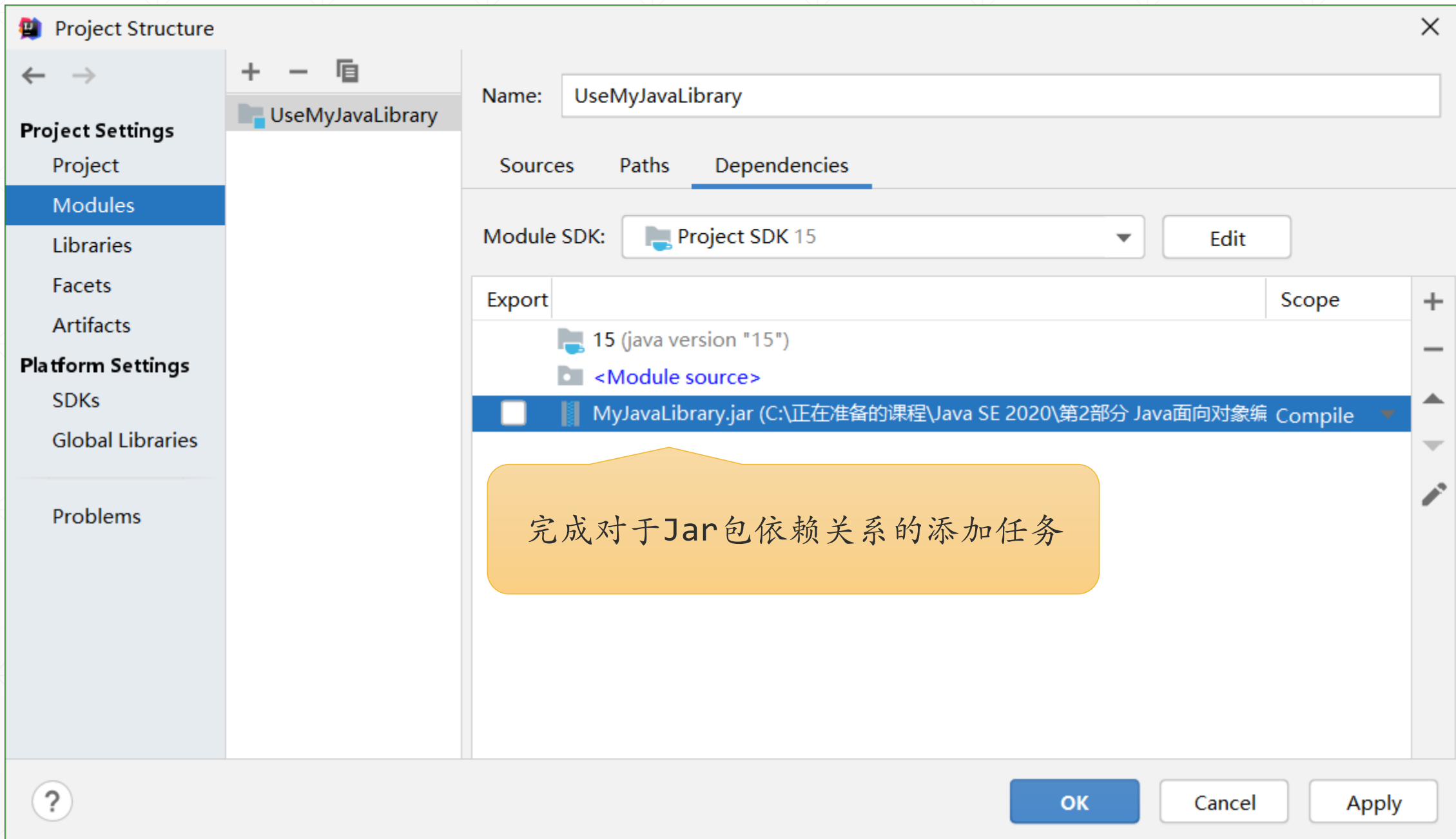
创建一个Java项目，取名UseMyLibrary。

创建用于保存外部jar包的文件夹



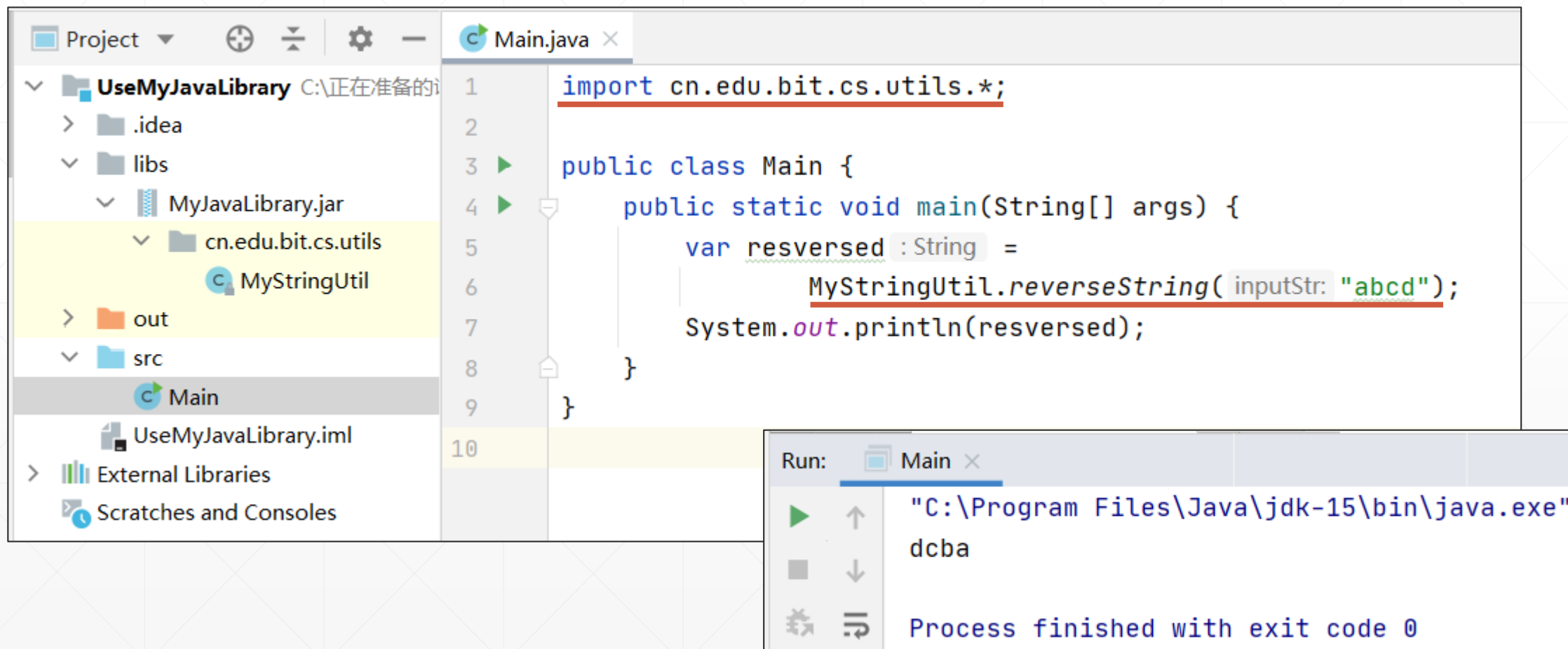
在项目根文件夹下创建一个lib子文件夹，将前面生成的jar包复制到此文件夹下。





现在可以写代码使用jar包中的类了

Main类使用jar包中的类，完成字符串倒序输出的功能：



小结



前面的示例，展示了一个完整的Java项目重构过程，分为两个阶段，第一阶段展示了如何抽取需要重用的代码生成jar包，第二阶段展示了如何在新项目中重用jar包。



对于一些简单的第三方组件，往往就是一个（或几个）jar包，只要按照本讲所介绍的内容，就可以很方便地在自己的项目中重用它们。



对于一些大型的组件或框架，往往包容N多个jar包，还采用这种手工添加jar包依赖的方式就行不通了，对于这种情况，必须使用构建工具，Maven和Gradle就是最著名的两个。

Java的组件仓库

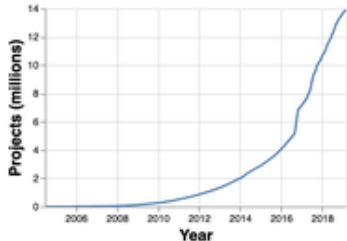
<https://mvnrepository.com/>



Search

[Categories](#) | [Popular](#) | [Contact Us](#)

Indexed Artifacts (18.3M)



Popular Categories

- Aspect Oriented
- Actor Frameworks
- Application Metrics
- Build Tools
- Bytecode Libraries
- Command Line Parsers
- Cache Implementations
- Cloud Computing
- Code Analyzers

What's New in Maven



Cloud Dataproc API V1 Rev150 1.25.0 4 usages
[com.google.apis](#) » [google-api-services-dataproc](#) » [v1beta2-rev20...](#) [Apache](#)

Cloud Dataproc API V1 Rev150 1.25.0
Last Release on Mar 12, 2020



Scala3 Library Bootstrapped 6 usages
[org.scala-lang](#) » [scala3-library_3.0.0-M2](#) » [3.0.0-M2-bin-...](#) [Apache](#)

scala3-library-bootstrapped
Last Release on Nov 8, 2020



Scala3 Compiler Bootstrapped 5 usages
[org.scala-lang](#) » [scala3-compiler_3.0.0-M2](#) » [3.0.0-M2-bin-...](#) [Apache](#)



Limited time offer: Get 10 free Adobe Stock images.

[ADS VIA CARBON](#)

Indexed Repositories (1288)

- Central
- Sonatype
- Spring Plugins
- Spring Lib M
- Hortonworks

搜索Gson

MVNREPOSITORY

Repository

- Central 290
- JCenter 97
- Sonatype 93
- Spring Plugins 49
- Spring Lib M 47
- ICM 11
- IBiblio 9
- Spring Lib Release 9

Group

- com.github 47
- org.apache 14

Found 434 results

Sort: **relevance** | popular | newest



1. **Gson**13,695 usages

com.google.code.gson » **gson**Apache

Gson

Last Release on Oct 4, 2019



2. **Converter: Gson**2,390 usages

com.squareup.retrofit2 » converter-gsonApache

A Retrofit Converter which uses Gson for serialization.

Last Release on May 20, 2020

点击这里.....

选择版本:

Home » [com.google.code.gson](#) » [gson](#)

**Gson**
Gson

License	Apache 2.0
Categories	JSON Libraries
Tags	google json
Used By	13,695 artifacts

[Central \(29\)](#) [Atlassian 3rd-P Old \(4\)](#) [Spring Plugins \(4\)](#)
[WSO2 Dist \(1\)](#) [Redhat GA \(4\)](#) [Redhat EA \(1\)](#) [ICM \(2\)](#)

Version	Repository	Usages	Date
2.8.6	Central	1,691	Oct, 2019
2.8.5	Central	3,644	May, 2018
2.8.4	Central	239	May, 2018
2.8.x 2.8.3	Central	24	Apr, 2018

下载jar包

Home » [com.google.code.gson](#) » [gson](#) » [2.8.6](#)

**Gson » 2.8.6**
Gson

License	Apache 2.0
Categories	JSON Libraries
Date	(Oct 04, 2019)
Files	jar (234 KB) View All
Repositories	Central WSO2 Public
Used By	13,695 artifacts

[Maven](#) [Gradle](#) [SBT](#) [Ivy](#) [Grape](#) [Leiningen](#) [Buildr](#)

```
<!-- https://mvnrepository.com/artifact
/com.google.code.gson/gson -->
<dependency>
  <groupId>com.google.code.gson</groupId>
  <artifactId>gson</artifactId>
  <version>2.8.6</version>
</dependency>
```

☒ Include comment with link to declaration

作业



Google的Gson是一个著名的Json组件库，它可以很方便地把一个Java对象转换为json字符串，也能读入一个json字符串，然后生成一个Java对象。

请使用本讲所介绍的方法，从MvnRepository网站下载Gson的jar包，然后在自己的项目中添加对它的依赖，然后，写测试代码测试Java对象与Json字符串之间的相互转换。

提示：百度一下，可以找到很多Gson相关的资料。



补充：谈谈Jar文件

什么是Jar包?

- Jar文件其实是一个压缩包，遵循于Zip数据压缩标准，可以使用WinRAR等解压软件打开。
- Jar文件中包容一个清单（manifest）文件，包容一些重要信息。比如可以在windows资源管理器中“双击”执行的jar包，其清单文件必须指明包中哪个类是“主类（main class）”，从而让JVM知道应该从哪个类中的main方法开始执行。

了解Jar命令

在命令提示符窗口中输入jar --help，可以查看它的使用文档。



```
Windows PowerShell
PS C:\> jar --help
用法: jar [OPTION...] [ [--release VERSION] [-C dir] files] ...
jar 创建类和资源的档案，并且可以处理档案中的
单个类或资源或者从档案中还原单个类或资源。

示例:
# 创建包含两个类文件的名为 classes.jar 的档案:
jar --create --file classes.jar Foo.class Bar.class
# 使用现有的清单创建档案，其中包含 foo/ 中的所有文件:
jar --create --file classes.jar --manifest mymanifest -C foo/ .
# 创建模块化 jar 档案，其中模块描述符位于
# classes/module-info.class:
jar --create --file foo.jar --main-class com.foo.Main --module-version 1.0
-C foo/ classes resources
# 将现有的非模块化 jar 更新为模块化 jar:
jar --update --file foo.jar --main-class com.foo.Main --module-version 1.0
-C foo/ module-info.class
# 创建包含多个发行版的 jar，并将一些文件放在 META-INF/versions/9 目录中:
jar --create --file mr.jar -C foo classes --release 9 -C foo9 classes

要缩短或简化 jar 命令，可以在单独的文本文件中指定参数，
并使用 @ 符号作为前缀将此文件传递给 jar 命令。

示例:
```

常用的Jar命令示例

- `jar cvf test.jar test`

将当前文件夹下test子文件夹的所有内容打包为test.jar

- `jar tf test.jar`

查看jar文件内容

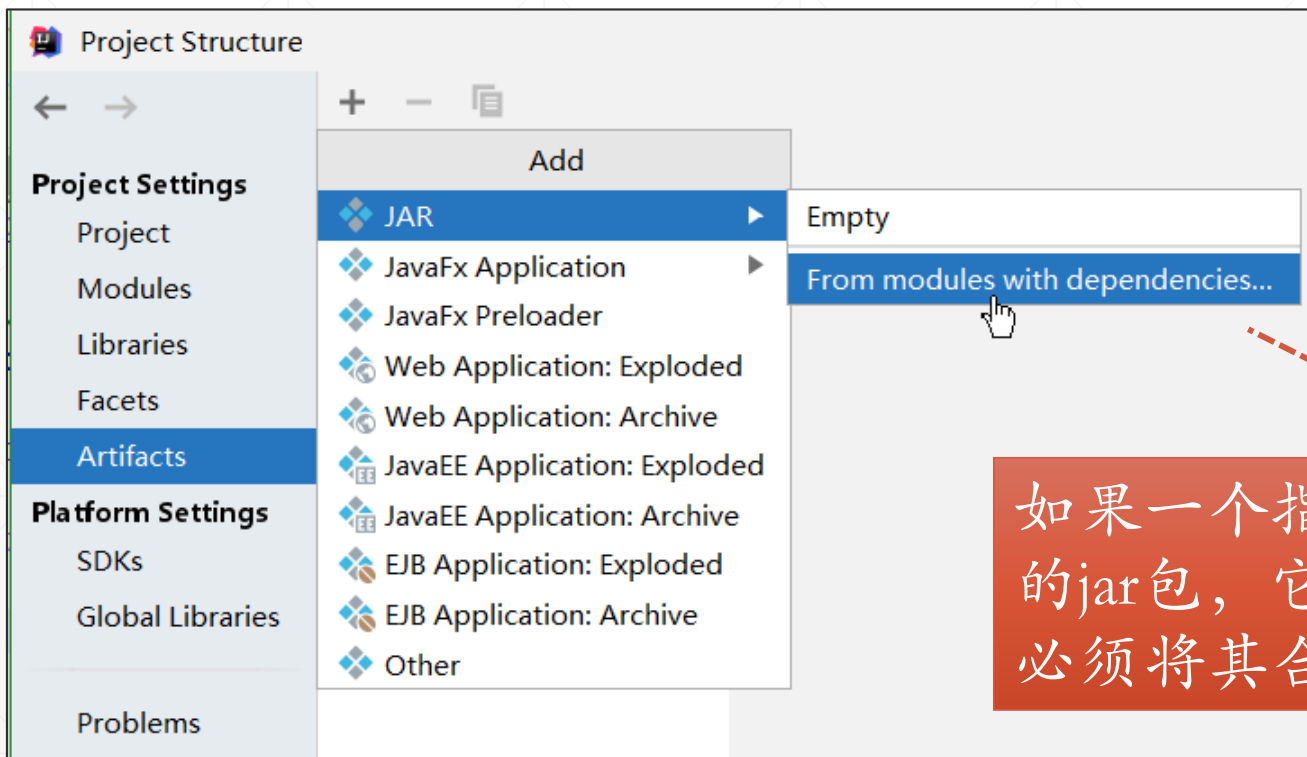
- `jar xvf test.jar`

解压缩jar包

创建可执行的jar包-1

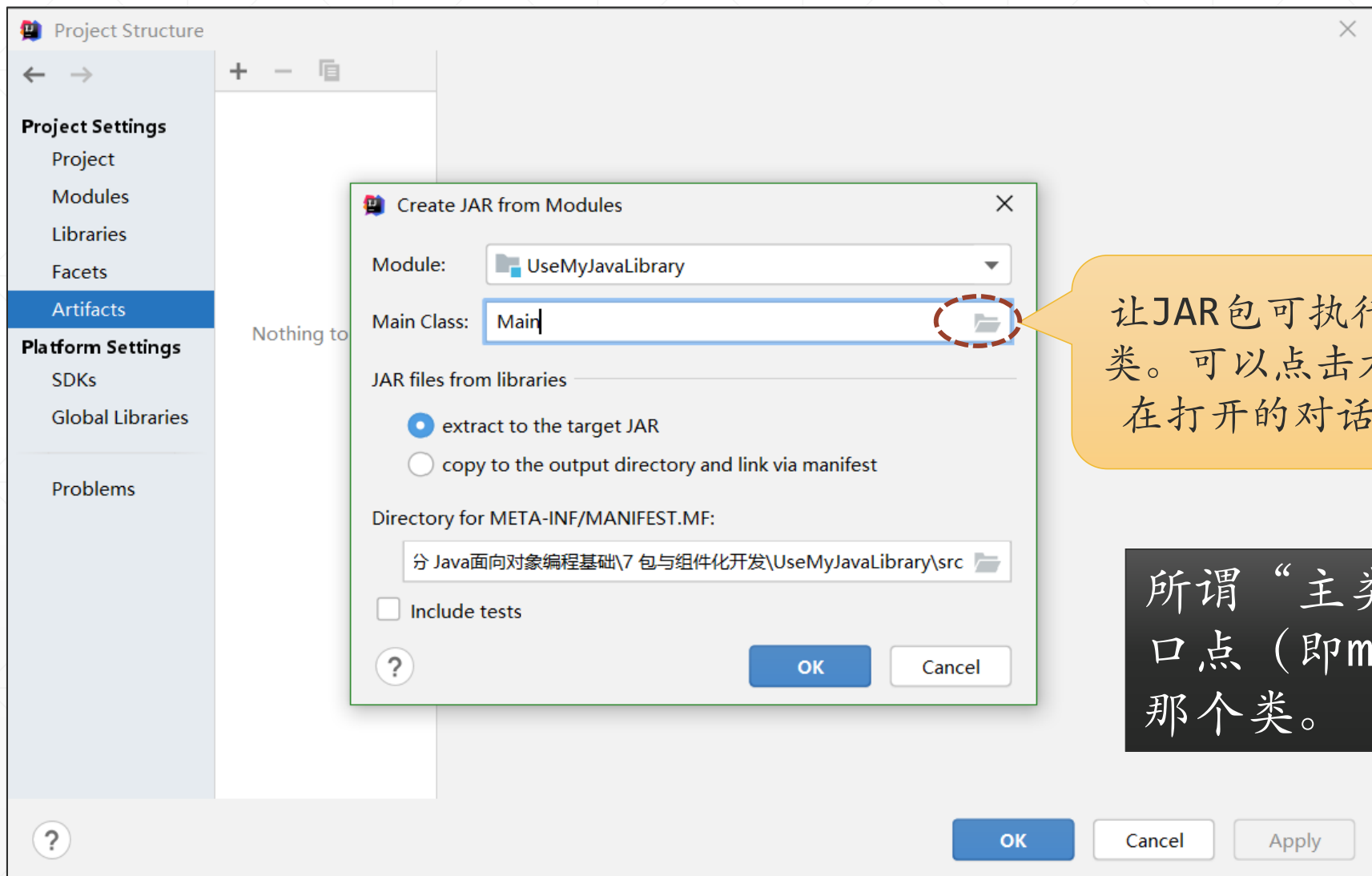


在前面介绍生成jar包时，曾经说过可以指定主类（Main Class），在Windows资源管理器中，双击这个jar包，就能直接运行它。但这有个前提条件，那就是这个jar包必须包容所有所需的组件。



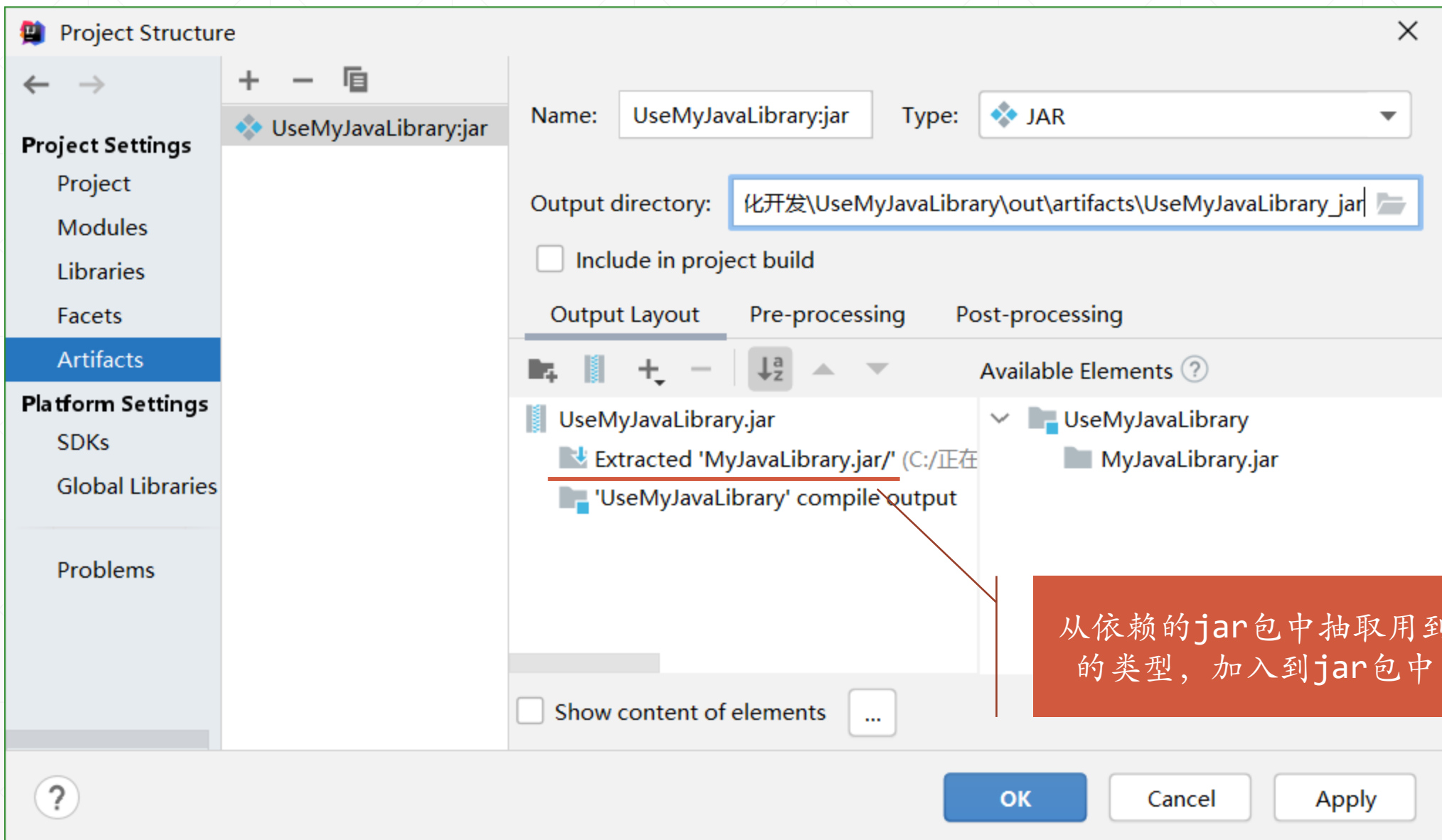
如果一个指定了主类（Main Class）的jar包，它依赖于另外的jar包，则必须将其合并起来，才能正确运行。

创建可执行的Jar包-2

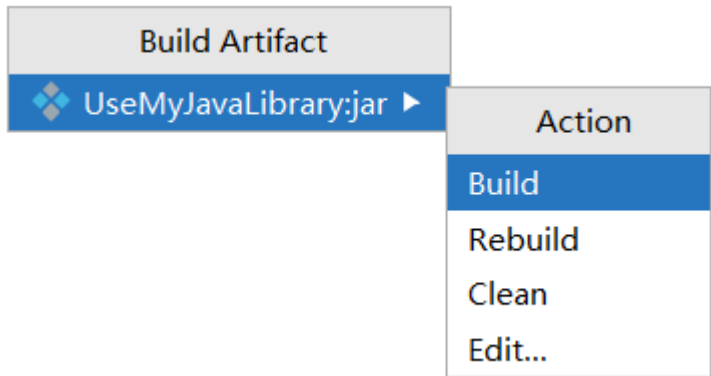


让JAR包可执行，关键在于指定主类。可以点击右侧的文件夹图标，在打开的对话框中选择主类。

所谓“主类”，就是程序入口点（即main方法）所在的那个类。

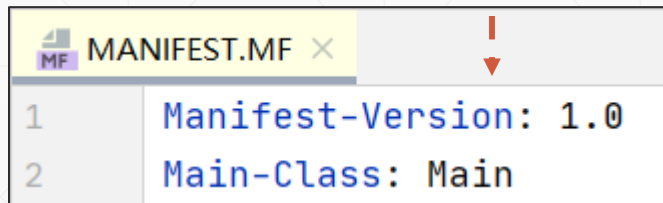
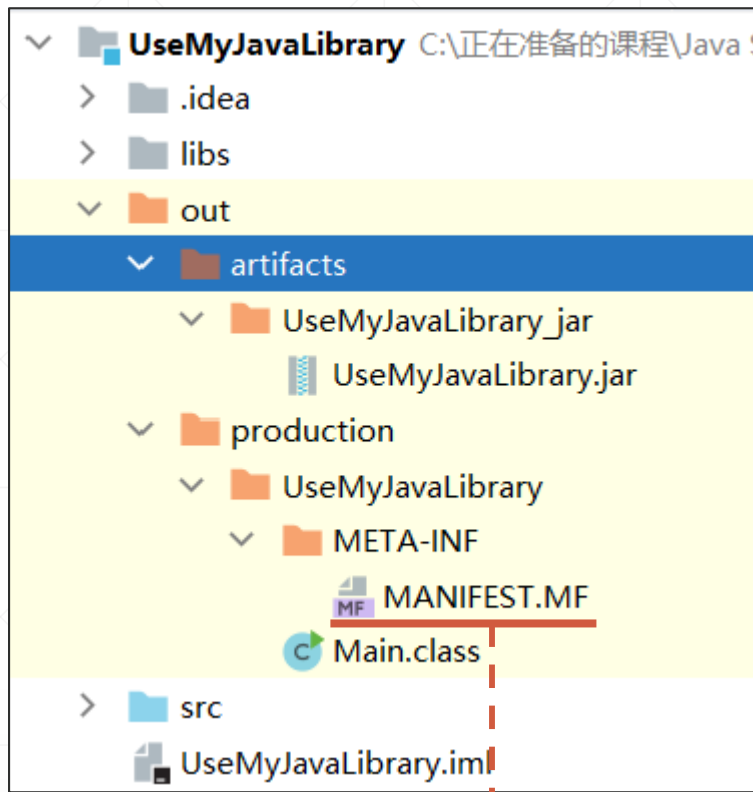


可执行Jar包的生成

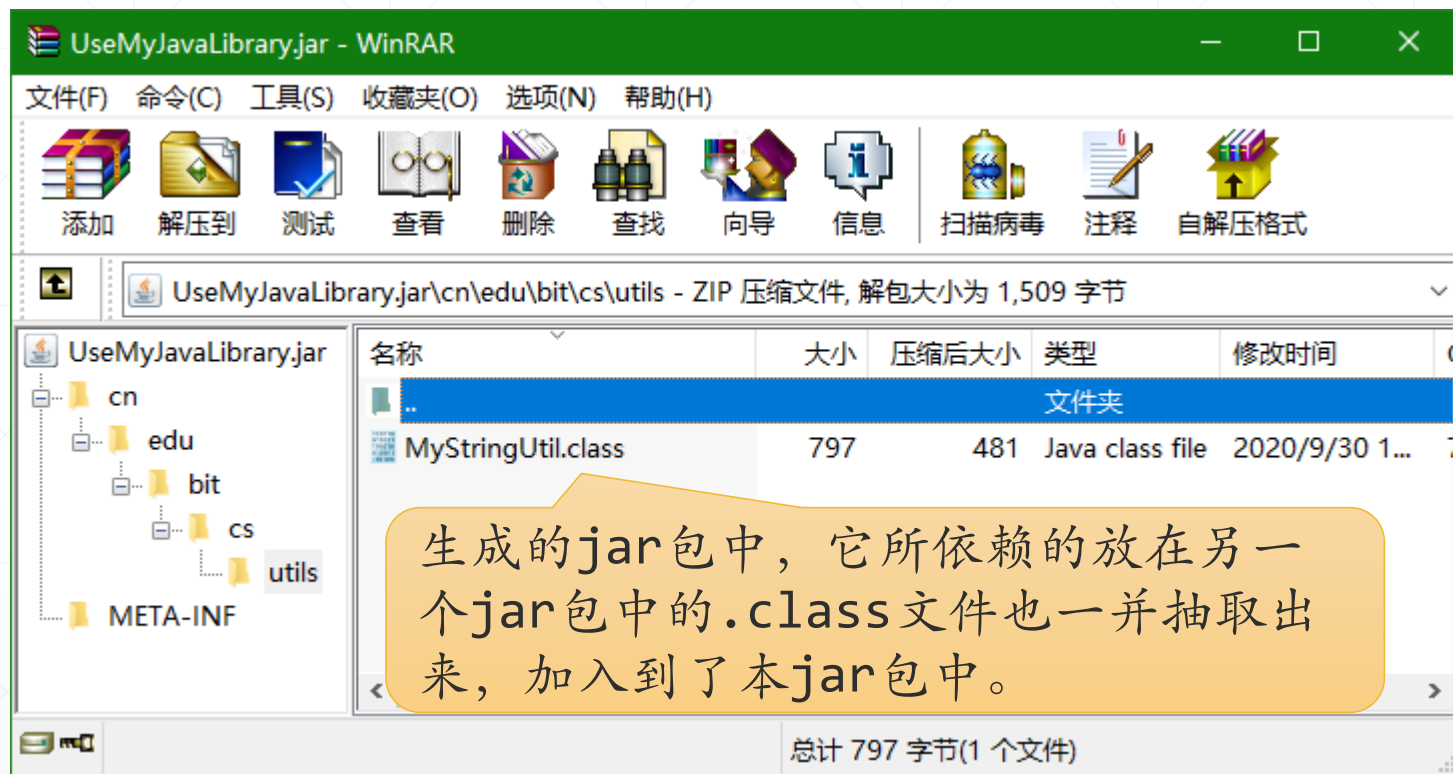


在artifacts文件夹下可以找到生成的jar包

在production文件夹下可以看到生成的jar包的内部结构



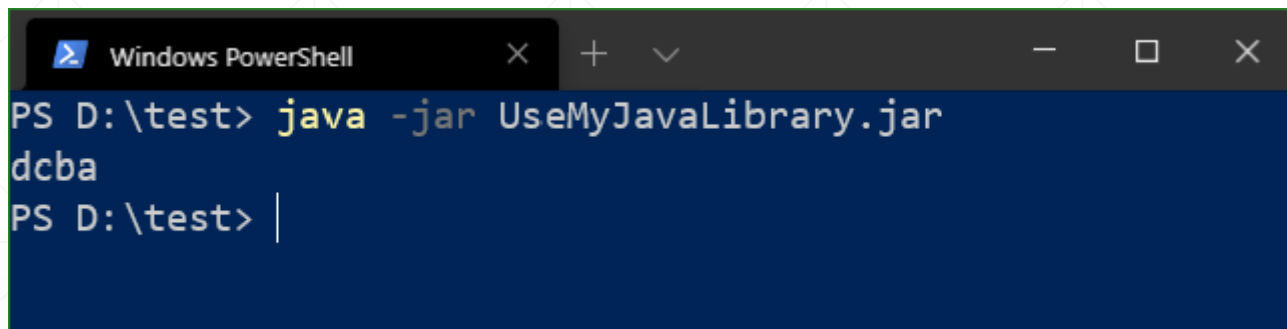
生成的jar包，不再依赖于其他的jar包！



由于UseMyJavaLibrary.jar中“集成了”它所用到的MyStringUtil类，所以，原先存放这个类的MyJavaLibrary.jar就不需要了。

使用命令行运行Runnable JAR包

使用java的-jar参数，可以直接运行一个“可运行的”JAR包。



```
Windows PowerShell
PS D:\test> java -jar UseMyJavaLibrary.jar
dcba
PS D:\test> |
```

Java组件化开发技能训练



用Java编写一个MathOpt类，其中使用Java提供的Math类，自己编写一些代码，封装一些数学函数功能，把此类放入mylib包中，打包生成jar包。



再编写另外一个项目，使用上面的MathOpt类完成一些数学计算工作，然后为其生成一个可执行的jar包。